

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

**ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN**



ALGORITMOS Y ESTRUCTURA DE DATOS

**ACTIVIDAD 2: REGISTRO, VALIDACIÓN Y CÁLCULO DE
PROMEDIO DE PRÁCTICAS EN JAVA**

ALUMNO:

➤ **VEGA MUÑOZ JHONSTYN YERAY**

DOCENTE:

➤ **FERNANDEZ BEJARANO RAUL ENRIQUE**

CICLO:

➤ **IV**

SECCIÓN:

➤ **A1**

HUANCAYO - PERÚ

2026

1. INTRODUCCIÓN

La presente actividad desarrolla una solución en Java, implementada en Apache NetBeans, para registrar y almacenar las notas de las prácticas de un alumno. El programa utiliza un arreglo de enteros para conservar las calificaciones y separa la solución en dos clases: Practicas, que contiene los datos y los métodos de procesamiento, y AplicaPracticas, que controla la ejecución mediante un menú de opciones.

El problema exige calcular el promedio de las prácticas eliminando la nota más baja, mostrar el promedio redondeado y validar los datos ingresados. Por ese motivo, la solución incorpora validaciones tanto para la cantidad mínima de prácticas como para el rango permitido de las notas, evitando valores menores que 0 o mayores que 20. Las capturas incluidas muestran el código desarrollado y las pruebas de ejecución realizadas.

2. ENUNCIADO DE LA ACTIVIDAD

La Actividad 2 solicita desarrollar una solución que permita ingresar y almacenar las notas de todas las prácticas de un alumno, determinar el promedio de las prácticas sabiendo que se elimina la práctica con la nota más baja, mostrar el promedio redondeado y validar el ingreso de datos.

ACTIVIDAD 2

1. Desarrollar una solución que permita ingresar y almacenar las notas de todas las prácticas de un alumno, determinar el promedio de las prácticas sabiendo que se elimina la práctica con la nota más baja, mostrar el promedio redondeado, validar el ingreso de datos.

El enunciado reúne cuatro necesidades principales: almacenar varias notas, controlar que los datos sean válidos, identificar la calificación mínima y excluirla del cálculo del promedio, y finalmente presentar el resultado redondeado. La implementación se diseñó de manera sencilla para que cada necesidad pueda reconocerse claramente dentro del código.

3. OBJETIVOS

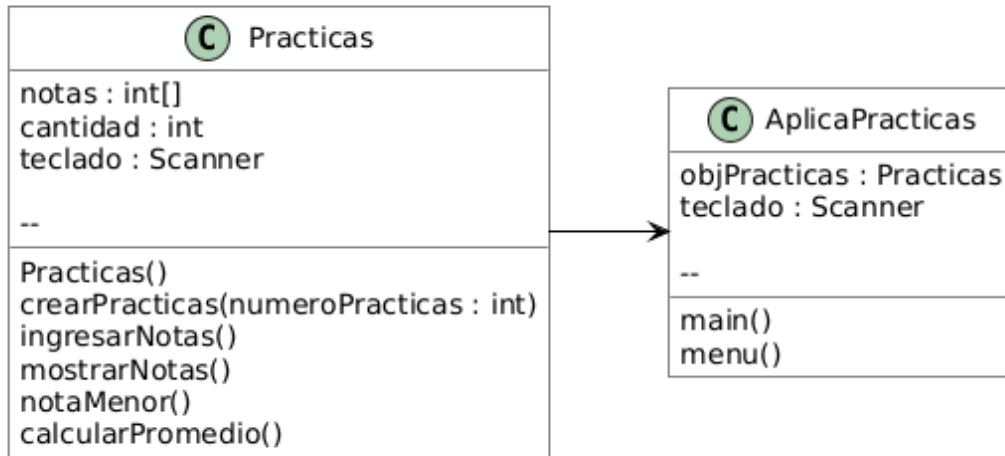
Objetivo general. Desarrollar una aplicación en Java que permita registrar, validar y procesar las notas de las prácticas de un alumno utilizando arreglos y métodos independientes.

Objetivos específicos. Crear un arreglo de notas según la cantidad de prácticas; validar que las calificaciones se encuentren entre 0 y 20; mostrar las notas almacenadas; identificar la nota menor; calcular el promedio excluyendo una vez la menor calificación; redondear el resultado; y comprobar el funcionamiento mediante pruebas de ejecución.

4. TDA Y ORGANIZACIÓN DE CLASES

El TDA de la Actividad 2 conserva la misma organización utilizada en el trabajo anterior. La clase Practicas concentra el arreglo de notas, la cantidad de prácticas, el Scanner y los métodos encargados de crear el registro, ingresar datos, mostrarlos, hallar la nota menor y calcular el promedio. La clase AplicaPracticas mantiene el método main(), el menú y el objeto objPracticas utilizado para llamar a los métodos de la clase principal de procesamiento.

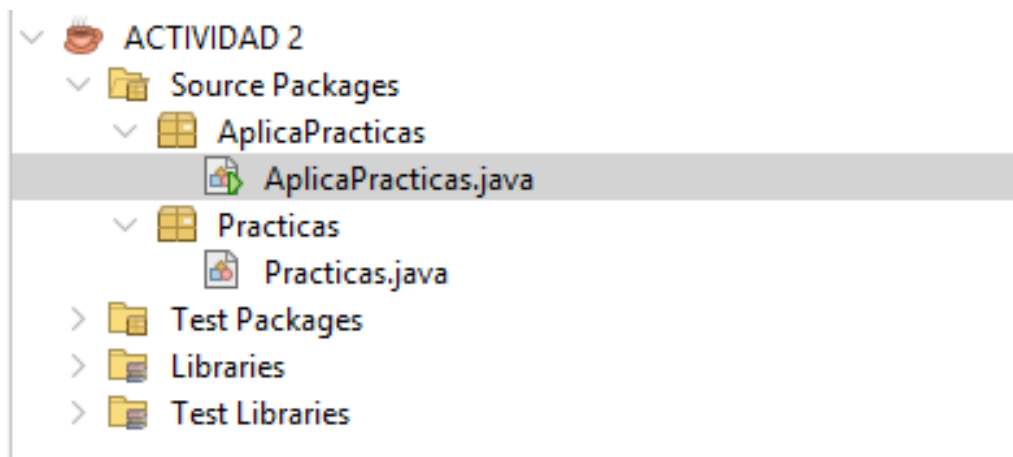
TDA - ACTIVIDAD 2



La separación entre ambas clases permite distinguir la interacción con el usuario de la lógica de procesamiento. **AplicaPracticas** recibe las opciones y datos generales, mientras que **Practicas** administra directamente el arreglo de notas y realiza los cálculos solicitados.

5. ESTRUCTURA DEL PROYECTO EN APACHE NETBEANS

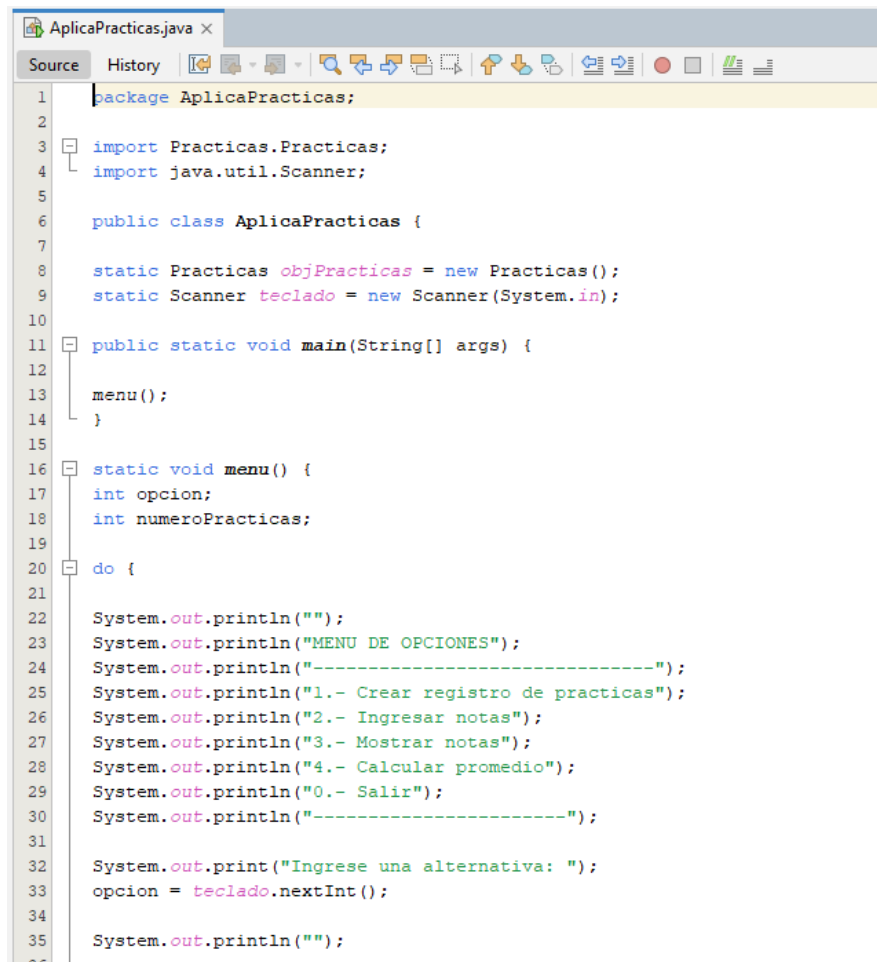
El proyecto se encuentra dividido en dos paquetes. El paquete **AplicaPracticas** contiene **AplicaPracticas.java**, donde se encuentra el método `main()` y el menú. El paquete **Practicas** contiene **Practicas.java**, donde se almacena el arreglo de notas y se implementan los métodos necesarios para resolver el enunciado.



6. DESARROLLO Y EXPLICACIÓN DEL CÓDIGO

6.1. Clase **AplicaPracticas**: inicio y menú de opciones

La clase **AplicaPracticas** importa la clase **Practicas** y **Scanner**. Se crea el objeto `objPracticas = new Practicas()`, que permite utilizar desde el menú los métodos implementados en el otro paquete. El método `main()` llama a `menu()`, por lo que toda la interacción comienza desde esa función. Dentro de `menu()` se declaran `opcion` y `numeroPracticas`, y un ciclo `do-while` mantiene el programa activo hasta que el usuario seleccione 0.

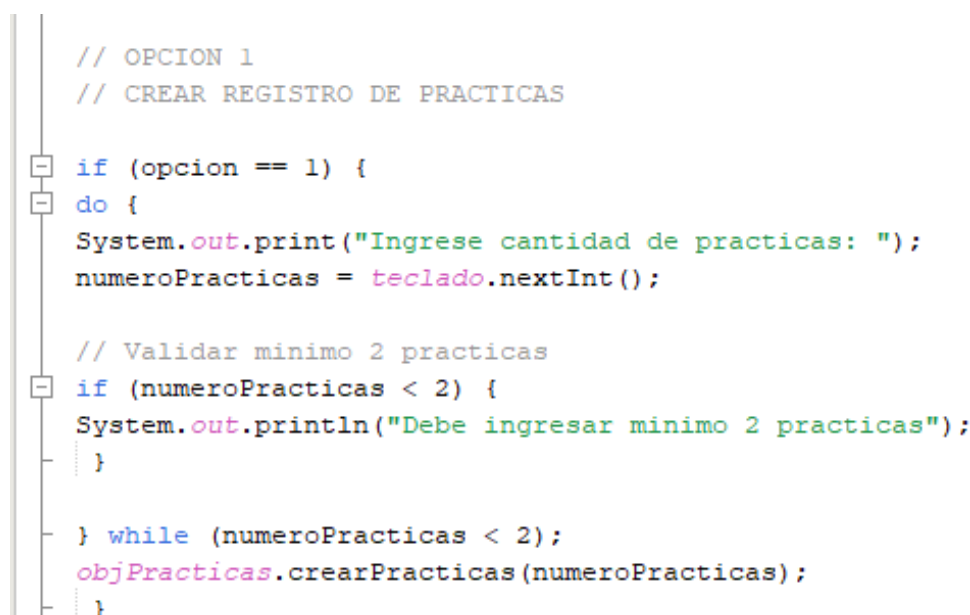


```
1 package AplicaPracticas;
2
3 import Practicas.Practicas;
4 import java.util.Scanner;
5
6 public class AplicaPracticas {
7
8     static Practicas objPracticas = new Practicas();
9     static Scanner teclado = new Scanner(System.in);
10
11     public static void main(String[] args) {
12
13         menu();
14     }
15
16     static void menu() {
17         int opcion;
18         int numeroPracticas;
19
20         do {
21
22             System.out.println("");
23             System.out.println("MENU DE OPCIONES");
24             System.out.println("-----");
25             System.out.println("1.- Crear registro de practicas");
26             System.out.println("2.- Ingresar notas");
27             System.out.println("3.- Mostrar notas");
28             System.out.println("4.- Calcular promedio");
29             System.out.println("0.- Salir");
30             System.out.println("-----");
31
32             System.out.print("Ingrese una alternativa: ");
33             opcion = teclado.nextInt();
34
35             System.out.println("");
36         } while (opcion != 0);
37     }
38 }
```

El menú presenta solo las operaciones necesarias para el enunciado: crear el registro, ingresar notas, mostrarlas y calcular el promedio. Esta organización evita agregar operaciones que no forman parte de la actividad y permite relacionar cada opción directamente con un método específico de la clase Practicas.

6.2. Opción 1: crear el registro y validar la cantidad de prácticas

La opción 1 solicita cuántas prácticas se van a registrar. Se utiliza un ciclo do-while porque el programa necesita garantizar que exista como mínimo una cantidad suficiente de notas para poder eliminar la menor y calcular un promedio con las restantes. En esta solución se exige un mínimo de dos prácticas; si el usuario ingresa un valor menor, se muestra un mensaje y se vuelve a pedir el dato.



```
// OPCION 1
// CREAR REGISTRO DE PRACTICAS

if (opcion == 1) {
    do {
        System.out.print("Ingrese cantidad de practicas: ");
        numeroPracticas = teclado.nextInt();

        // Validar minimo 2 practicas
        if (numeroPracticas < 2) {
            System.out.println("Debe ingresar minimo 2 practicas");
        }

    } while (numeroPracticas < 2);
    objPracticas.crearPracticas(numeroPracticas);
}
```

Cuando `numeroPracticas` es válido, `objPracticas.crearPracticas(numeroPracticas)` envía la cantidad a la clase `Practicas`. Esta separación es importante porque `AplicaPracticas` se encarga de leer la opción y `Practicas` se encarga de crear realmente el arreglo que almacenará las notas.

6.3. Opción 2: ingresar las notas

La opción 2 no necesita leer cada nota directamente en la clase principal. Su única responsabilidad es llamar a `objPracticas.ingresarNotas()`. De esta manera, la validación y almacenamiento de las calificaciones quedan concentrados en la clase `Practicas`, evitando duplicar lógica en el menú.

```
// OPCION 2
// INGRESAR LAS NOTAS

if (opcion == 2) {
    objPracticas.ingresarNotas();
}
```

6.4. Opción 3: mostrar las notas almacenadas

La opción 3 llama a `mostrarNotas()`. El propósito es comprobar que las calificaciones fueron almacenadas correctamente en el arreglo y ofrecer una salida ordenada con el número de práctica y su nota correspondiente.

```
// OPCION 3
// MOSTRAR LAS NOTAS

if (opcion == 3) {
    objPracticas.mostrarNotas();
}
```

6.5. Opción 4: calcular el promedio solicitado

La opción 4 ejecuta `calcularPromedio()`. No solicita datos adicionales porque el método utiliza directamente las notas que ya fueron registradas. La lógica completa para buscar la menor nota, excluirla del cálculo, obtener el promedio y redondearlo se encuentra en la clase `Practicas`.

```
// OPCION 4
// CALCULAR EL PROMEDIO

if (opcion == 4) {
    objPracticas.calcularPromedio();
}
```

6.6. Opción 0: finalización controlada

Cuando `opcion` es igual a 0 se muestra el mensaje “SALIENDO DEL PROGRAMA”. El ciclo `do-while` termina porque su condición de repetición es `opcion != 0`. Esta salida permite cerrar el programa de forma controlada y comprensible para el usuario.

```

73 // OPCION 0
74 // SALIR
75
76 if (opcion == 0) {
77     System.out.println("SALIENDO DEL PROGRAMA");
78 }
79 } while (opcion != 0);
80 }
81 }

```

6.7. Clase Practicas: atributos, creación del arreglo e ingreso validado

La clase Practicas declara notas como un arreglo de enteros, cantidad para recordar cuántas prácticas se registrarán y teclado para leer las calificaciones. El método crearPracticas(int numeroPracticas) reserva el espacio del arreglo con new int[numeroPracticas] y guarda la cantidad recibida. Esto permite adaptar el tamaño del arreglo al número de prácticas indicado por el usuario.

```

1 package Practicas;
2
3 import java.util.Scanner;
4
5 public class Practicas {
6
7     static int[] notas;
8     static int cantidad = 0;
9
10    static Scanner teclado = new Scanner(System.in);
11
12    // Constructor
13    public Practicas() {
14    }
15    // CREAR EL ARREGLO DE NOTAS
16
17    public void crearPracticas(int numeroPracticas) {
18
19        notas = new int[numeroPracticas];
20        cantidad = numeroPracticas;
21
22        System.out.println("Registro de practicas creado");
23    }
24    // INGRESAR Y VALIDAR LAS NOTAS
25
26    public void ingresarNotas() {
27        if (notas == null) {
28
29            System.out.println("Primero debe crear el registro de practicas");
30
31            return;
32        }
33
34        int nota;
35        for (int i = 0; i < cantidad; i++) {
36            do {
37
38                System.out.print("Ingrese nota de la practica "
39                    + (i + 1) + ": ");
40                nota = teclado.nextInt();
41                // Validar que la nota sea de 0 a 20
42
43                if (nota < 0 || nota > 20) {
44                    System.out.println("Nota incorrecta. Ingrese una nota de 0 a 20");
45                }
46
47            } while (nota < 0 || nota > 20);
48        }
49    }
50 }

```

Antes de ingresar notas se verifica si notas == null. Esta comprobación evita intentar guardar datos cuando todavía no se ha creado el arreglo. Luego un for recorre todas las posiciones y, dentro de cada iteración, un do-while solicita una calificación. La condición nota < 0 || nota > 20 obliga a repetir la lectura cuando el valor se encuentra fuera del rango válido de 0 a 20. Por ello la validación responde directamente a la exigencia del enunciado de controlar el ingreso de datos.

6.8. Almacenamiento, visualización y búsqueda de la nota menor

Después de validar una nota, la instrucción `notas[i] = nota` la almacena en la posición correspondiente del arreglo. Al terminar el recorrido se informa que las notas fueron registradas correctamente. El método `mostrarNotas()` comprueba primero que el arreglo exista y luego utiliza un `for` para imprimir cada práctica junto con su calificación.

```
49     notas[i] = nota;
50
51 }
52
53 System.out.println("Notas registradas correctamente");
54
55 }
56 // MOSTRAR LAS NOTAS INGRESADAS
57
58 public void mostrarNotas() {
59
60     if (notas == null) {
61         System.out.println("No existen notas registradas");
62         return;
63     }
64
65     System.out.println("");
66     System.out.println("LISTA DE NOTAS");
67     System.out.println("-----");
68
69     for (int i = 0; i < cantidad; i++) {
70
71         System.out.println("Practica " + (i + 1)
72             + ": " + notas[i]);
73     }
74
75     // BUSCAR LA NOTA MAS BAJA
76
77     public int notaMenor() {
78
79         int menor = notas[0];
80
81         for (int i = 1; i < cantidad; i++) {
82
83             if (notas[i] < menor) {
84                 menor = notas[i];
85             }
86         }
87
88         return menor;
89     }
90 }
```

El método `notaMenor()` inicia la variable `menor` con `notas[0]`. A partir de la segunda posición compara cada nota con el menor actual; cuando encuentra una calificación inferior, actualiza la variable. Al finalizar devuelve el valor mínimo. Se utiliza este procedimiento porque el promedio debe excluir exactamente una vez la práctica con la nota más baja.

6.9. Cálculo del promedio eliminando la menor nota y redondeo

El método `calcularPromedio()` primero verifica que existan notas y que la cantidad sea por lo menos dos. Después obtiene la nota mínima mediante `notaMenor()`, suma todas las calificaciones con un `for` y resta una sola vez el valor menor. Esta resta equivale a eliminar esa práctica únicamente para el cálculo, sin borrar físicamente el dato del arreglo.

```

92 // CALCULAR EL PROMEDIO
93 // ELIMINANDO LA NOTA MAS BAJA
94
95 public void calcularPromedio() {
96
97     if (notas == null) {
98         System.out.println("No existen notas registradas");
99         return;
100     }
101
102     if (cantidad < 2) {
103         System.out.println("Se necesitan minimo 2 practicas");
104         return;
105     }
106
107     int suma = 0;
108     int menor = notaMenor();
109     // Sumar todas las notas
110     for (int i = 0; i < cantidad; i++) {
111         suma = suma + notas[i];
112     }
113
114     // Restar una vez la nota mas baja
115     suma = suma - menor;
116
117     // Calcular promedio sin la nota mas baja
118     double promedio = (double) suma / (cantidad - 1);
119
120     // Redondear el promedio
121     long promedioRedondeado = Math.round(promedio);
122
123     System.out.println("");
124     System.out.println("Nota mas baja eliminada: " + menor);
125     System.out.println("Promedio: " + promedio);
126     System.out.println("Promedio redondeado: " + promedioRedondeado);
127 }
128 }

```

El promedio se calcula con $(\text{double}) \text{ suma} / (\text{cantidad} - 1)$. El cast a double evita una división entera y permite conservar decimales. El denominador $\text{cantidad} - 1$ representa las prácticas que quedan después de excluir la menor. Finalmente $\text{Math.round}(\text{promedio})$ produce el promedio redondeado solicitado y se muestran la nota eliminada, el promedio decimal y el promedio redondeado.

7. PRUEBAS DE EJECUCIÓN

7.1. Creación del registro e ingreso de notas

En la primera prueba se seleccionó la opción 1 y se creó un registro para cinco prácticas. Luego, mediante la opción 2, se ingresaron las notas 15, 18, 10, 16 y 19. Todas se encuentran dentro del rango permitido, por lo que el programa las aceptó y confirmó el registro.

```

run:

MENU DE OPCIONES
-----
1.- Crear registro de practicas
2.- Ingresar notas
3.- Mostrar notas
4.- Calcular promedio
0.- Salir
-----
Ingrese una alternativa: 1

Ingrese cantidad de practicas: 5
Registro de practicas creado

MENU DE OPCIONES
-----
1.- Crear registro de practicas
2.- Ingresar notas
3.- Mostrar notas
4.- Calcular promedio
0.- Salir
-----
Ingrese una alternativa: 2

Ingrese nota de la practica 1: 15
Ingrese nota de la practica 2: 18
Ingrese nota de la practica 3: 10
Ingrese nota de la practica 4: 16
Ingrese nota de la practica 5: 19
Notas registradas correctamente

MENU DE OPCIONES
-----
1.- Crear registro de practicas
2.- Ingresar notas
3.- Mostrar notas
4.- Calcular promedio
0.- Salir
-----
Ingrese una alternativa: 3

```

La misma ejecución muestra el acceso a la opción 3 para comprobar los datos almacenados. Esta verificación es importante porque permite observar que el arreglo conserva las cinco notas en el mismo orden en que fueron ingresadas.

7.2. Visualización, cálculo del promedio y validación de datos

Al mostrar las notas se obtiene la lista: práctica 1 = 15, práctica 2 = 18, práctica 3 = 10, práctica 4 = 16 y práctica 5 = 19. Al seleccionar la opción 4, el programa identifica 10 como la nota más baja. La suma de las cuatro notas restantes es 68 y, al dividir entre 4, el promedio obtenido es 17.0; Math.round() confirma un promedio redondeado de 17.

```

LISTA DE NOTAS
-----
Practica 1: 15
Practica 2: 18
Practica 3: 10
Practica 4: 16
Practica 5: 19

MENU DE OPCIONES
-----
1.- Crear registro de practicas
2.- Ingresar notas
3.- Mostrar notas
4.- Calcular promedio
0.- Salir
-----
Ingrese una alternativa: 4

Nota mas baja eliminada: 10
Promedio: 17.0
Promedio redondeado: 17

MENU DE OPCIONES
-----
1.- Crear registro de practicas
2.- Ingresar notas
3.- Mostrar notas
4.- Calcular promedio
0.- Salir
-----
Ingrese una alternativa: 2

Ingrese nota de la practica 1: 25
Nota incorrecta. Ingrese una nota de 0 a 20
Ingrese nota de la practica 1: 34
Nota incorrecta. Ingrese una nota de 0 a 20
Ingrese nota de la practica 1: 12
Ingrese nota de la practica 2: 11
Ingrese nota de la practica 3: 9
Ingrese nota de la practica 4: 5
Ingrese nota de la practica 5: 3
Notas registradas correctamente

```

En la misma evidencia se comprueba la validación: al intentar ingresar 25 y luego 34 para la práctica 1, el sistema rechaza ambos valores porque exceden el límite de 20 y vuelve a solicitar la nota. Cuando se ingresa 12, el valor es aceptado y el proceso continúa con las demás prácticas. Esta prueba demuestra que la condición del do-while impide almacenar calificaciones inválidas.

7.3. Salida del programa

Finalmente se seleccionó la opción 0. El programa mostró el mensaje de salida y Apache NetBeans indicó BUILD SUCCESSFUL, confirmando que la ejecución terminó sin errores de compilación ni fallos durante la prueba.

```

MENU DE OPCIONES
-----
1.- Crear registro de practicas
2.- Ingresar notas
3.- Mostrar notas
4.- Calcular promedio
0.- Salir
-----
Ingrese una alternativa: 0

SALIENDO DEL PROGRAMA
BUILD SUCCESSFUL (total time: 3 minutes 47 seconds)
|

```

8. RESULTADOS

La solución cumplió con el enunciado de la actividad. Se logró crear un registro de prácticas de tamaño variable, almacenar las notas en un arreglo, rechazar valores fuera del rango de 0 a 20, visualizar los datos ingresados, localizar la menor calificación y calcular el promedio sin considerar una vez dicha nota. En la prueba principal, con las notas 15, 18, 10, 16 y 19, la nota mínima fue 10 y el promedio de las cuatro notas restantes fue 17.0, con un resultado redondeado de 17.

También se comprobó el control de datos mediante los valores 25 y 34, los cuales fueron rechazados correctamente. La estructura de dos clases permitió mantener separadas la interacción del menú y las operaciones realizadas sobre las notas, facilitando la comprensión del programa.

9. CONCLUSIONES

La Actividad 2 permitió aplicar arreglos, ciclos for y do-while, condicionales, métodos y manejo básico de objetos en Java. El arreglo facilita almacenar varias notas bajo una misma estructura y recorrerlas para mostrarlas, sumarlas o compararlas.

La validación mediante do-while garantiza que las notas se mantengan dentro del rango académico de 0 a 20. Asimismo, el método `notaMenor()` permite identificar la calificación que debe excluirse, mientras que `calcularPromedio()` concentra la suma, la eliminación lógica de la menor nota, la división entre las prácticas restantes y el redondeo del resultado.

Finalmente, la separación entre `Practicas` y `AplicaPracticas` mejora el orden del código: una clase procesa los datos y la otra controla la ejecución. Las pruebas realizadas en Apache NetBeans confirmaron que el programa responde correctamente tanto con datos válidos como con intentos de ingreso inválidos.